```
import pandas as pd
```

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

Double-click (or enter) to edit

```
df = pd.read_csv('/content/drive/MyDrive/data/Advertising Budget and Sales.csv')

df.head()
```

	Unnamed: 0	TV Ad Budget ($)	Radio Ad Budget ($)	Newspaper Ad Budget ($)	Sales ($)
0	1	230.1	37.8	69.2	22.1
1	2	44.5	39.3	45.1	10.4
2	3	17.2	45.9	69.3	9.3
3	4	151.5	41.3	58.5	18.5
4	5	180.8	10.8	58.4	12.9

Next steps: Generate code with df New interactive sheet

Data Preperation

```
df = df.rename(columns={"Sales ($)": "Sales(Units)"})

df.columns
```

```
Index(['Unnamed: 0', 'TV Ad Budget ($)', 'Radio Ad Budget ($)',
       'Newspaper Ad Budget ($)', 'Sales(Units)'],
      dtype='object')
```

```
if "Unnamed: 0" in df.columns:
    df = df.drop(columns=["Unnamed: 0"])

print("Missing Values:")
print(df.isnull().sum())

df = df.drop_duplicates()

df.info()
```

```
Missing Values:
TV Ad Budget ($)           0
Radio Ad Budget ($)        0
Newspaper Ad Budget ($)    0
Sales(Units)               0
dtype: int64
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 4 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   TV Ad Budget ($)         200 non-null    float64
 1   Radio Ad Budget ($)      200 non-null    float64
 2   Newspaper Ad Budget ($)  200 non-null    float64
 3   Sales(Units)             200 non-null    float64
dtypes: float64(4)
memory usage: 6.4 KB
```

Data Engineering

```
# Creating total advertising budget column
```

```
df["Total_Ad_Budget"] = (
    df["TV Ad Budget ($)"] +
    df["Radio Ad Budget ($)"] +
    df["Newspaper Ad Budget ($)"]
)

df = df.round(1)

df.head()
```

	TV Ad Budget ($)	Radio Ad Budget ($)	Newspaper Ad Budget ($)	Sales(Units)	Total_Ad_Budget
0	230.1	37.8	69.2	22.1	337.1
1	44.5	39.3	45.1	10.4	128.9
2	17.2	45.9	69.3	9.3	132.4
3	151.5	41.3	58.5	18.5	251.3
4	180.8	10.8	58.4	12.9	250.0

Next steps: Generate code with df New interactive sheet

this column will allow me to see what each company is spending on all three ad budgets. this is probably the most important added column because of the amount of things I need to know what the total for all of these ad budgets.

```
# creating a channel share columns for each ad

df["TV_Share(%)"] = df["TV Ad Budget ($)"] / df["Total_Ad_Budget"]
df["Radio_Share(%)"] = df["Radio Ad Budget ($)"] / df["Total_Ad_Budget"]
df["Newspaper_Share(%)"] = df["Newspaper Ad Budget ($)"] / df["Total_Ad_Budget"]

# round share columns and make them percentages

share_cols = ["TV_Share(%)", "Radio_Share(%)", "Newspaper_Share(%)"]

df[share_cols] = (df[share_cols] * 100).round(2)

df.head()
```

	TV Ad Budget ($)	Radio Ad Budget ($)	Newspaper Ad Budget ($)	Sales(Units)	Total_Ad_Budget	TV_Share(%)	Radio_Share(%)	Newspaper_Share(%)
0	230.1	37.8	69.2	22.1	337.1	68.26	11.21	20.53
1	44.5	39.3	45.1	10.4	128.9	34.52	30.49	34.99
2	17.2	45.9	69.3	9.3	132.4	12.99	34.67	52.34

Next steps: Generate code with df New interactive sheet

These three columns will help me see which ad budget has the most money invested in them, which will help me with answering my specific question which channel has the most influence on volume?

```
# making a sales effciency column

df["Sales_per_Dollar"] = df["Sales(Units)"] / df["Total_Ad_Budget"]

df.head()
```

	TV Ad Budget ($)	Radio Ad Budget ($)	Newspaper Ad Budget ($)	Sales(Units)	Total_Ad_Budget	TV_Share(%)	Radio_Share(%)	Newspaper_Share(%)	Sales_per
0	230.1	37.8	69.2	22.1	337.1	68.26	11.21	20.53	
1	44.5	39.3	45.1	10.4	128.9	34.52	30.49	34.99	
2	17.2	45.9	69.3	9.3	132.4	12.99	34.67	52.34	
3	151.5	41.3	58.5	18.5	251.3	60.29	16.43	23.28	

Next steps: Generate code with df New interactive sheet

This will help me with answer one of my specific questions, what is the marginal increase in units sold for every $10,000 spent?

```
new_order = [
    "TV Ad Budget ($)",
    "Radio Ad Budget ($)",
    "Newspaper Ad Budget ($)",
    "TV_Share(%)",
    "Radio_Share(%)",
    "Newspaper_Share(%)",
    "Sales_per_Dollar",
    "Total_Ad_Budget",
    "Sales(Units)"
]

df = df[new_order]

df.head()
```

	TV Ad Budget ($)	Radio Ad Budget ($)	Newspaper Ad Budget ($)	TV_Share(%)	Radio_Share(%)	Newspaper_Share(%)	Sales_per_Dollar	Total_Ad_Budget	Sales
0	230.1	37.8	69.2	68.26	11.21	20.53	0.065559	337.1	
1	44.5	39.3	45.1	34.52	30.49	34.99	0.080683	128.9	
2	17.2	45.9	69.3	12.99	34.67	52.34	0.070242	132.4	
3	151.5	41.3	58.5	60.29	16.43	23.28	0.073617	251.3	

Next steps: Generate code with df New interactive sheet

I had to reorder the columns because it makes the dataset easier to read and it looks a lot more organized when it is order like this.

```
df.to_csv("f_clean_advertising.csv", index=False)
```

```
from google.colab import files
files.download("f_clean_advertising.csv")
```

Exploratory analysis

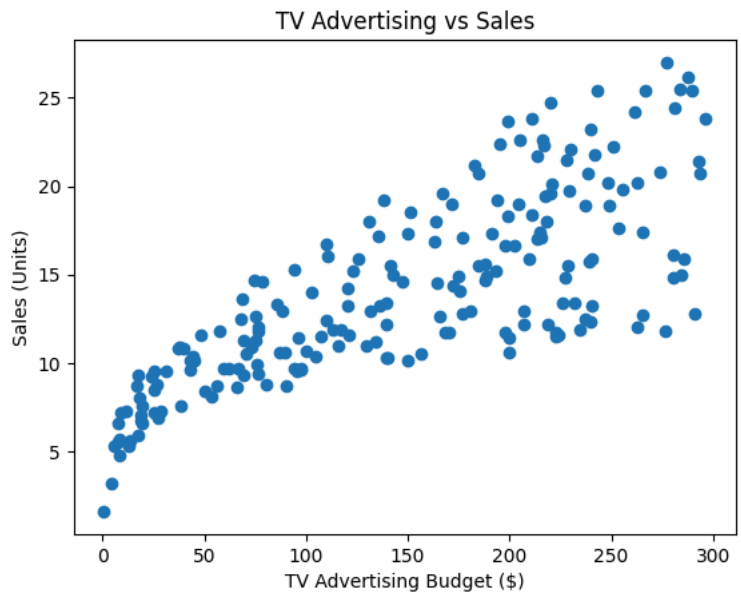
```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("f_clean_advertising.csv")

df.head()
```

	TV Ad Budget ($)	Radio Ad Budget ($)	Newspaper Ad Budget ($)	TV_Share(%)	Radio_Share(%)	Newspaper_Share(%)	Sales_per_Dollar	Total_Ad_Budget	Sales
0	230.1	37.8	69.2	68.26	11.21	20.53	0.065559	337.1	
1	44.5	39.3	45.1	34.52	30.49	34.99	0.080683	128.9	
2	17.2	45.9	69.3	12.99	34.67	52.34	0.070242	132.4	
3	151.5	41.3	58.5	60.29	16.43	23.28	0.073617	251.3	

Next steps: Generate code with df New interactive sheet

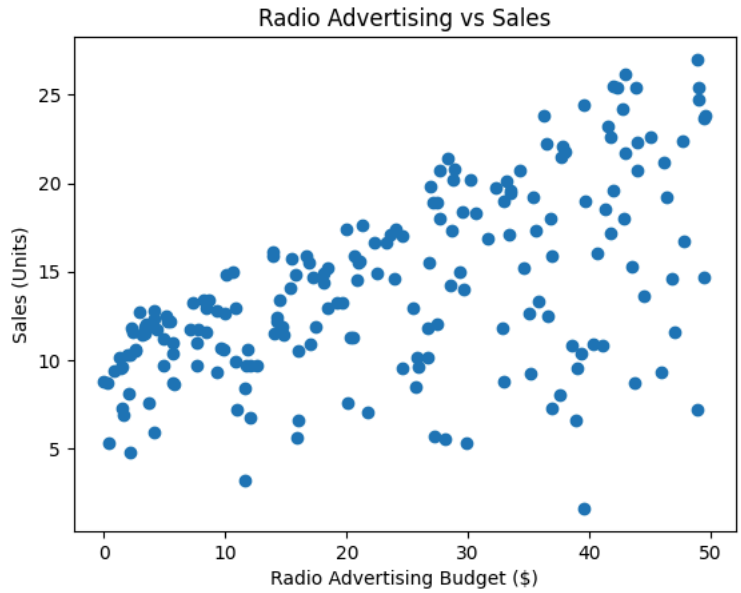
TV advirtising Vs. Sales

```
plt.figure()

plt.scatter(df["TV Ad Budget ($)"], df["Sales(Units)"])

plt.xlabel("TV Advertising Budget ($)")
plt.ylabel("Sales (Units)")
plt.title("TV Advertising vs Sales")

plt.show()
```



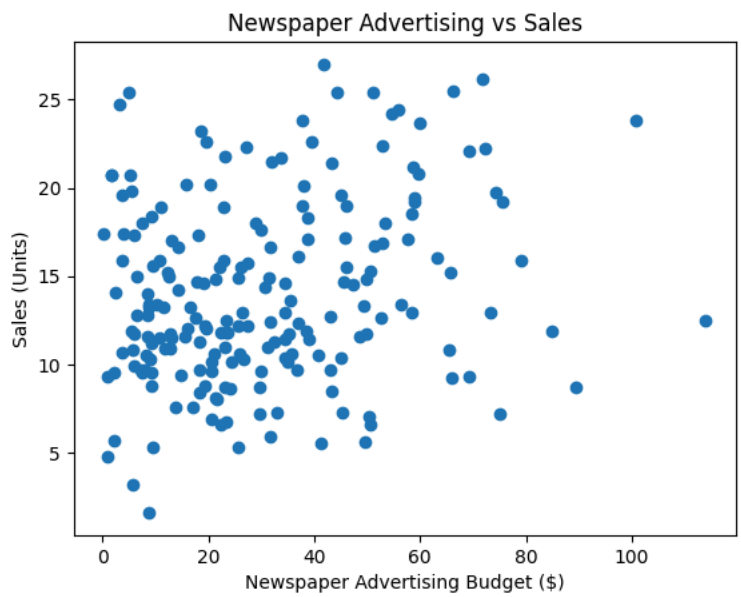
Radio Advirtising Vs. Sales

```
plt.figure()

plt.scatter(df["Radio Ad Budget ($)"], df["Sales(Units)"])

plt.xlabel("Radio Advertising Budget ($)")
plt.ylabel("Sales (Units)")
plt.title("Radio Advertising vs Sales")

plt.show()
```



Newspaper advirtising Vs. Sales

```
plt.figure()

plt.scatter(df["Newspaper Ad Budget ($)"], df["Sales(Units)"])

plt.xlabel("Newspaper Advertising Budget ($)")
plt.ylabel("Sales (Units)")
plt.title("Newspaper Advertising vs Sales")

plt.show()
```



explanation:

The exploratory analysis suggests that TV advertising has the strongest relationship with sales, followed by radio advertising. Newspaper advertising appears to have a weaker relationship with sales. This suggests that TV and radio advertising may be more important predictors when building a sales prediction model.

Tv spend level

```
df["TV_spend_level"] = pd.qcut(
    df["TV Ad Budget ($)"],
    q=3,
    labels=["Low", "Medium", "High"]
)

df.head()
```

	TV Ad Budget ($)	Radio Ad Budget ($)	Newspaper Ad Budget ($)	TV_Share(%)	Radio_Share(%)	Newspaper_Share(%)	Sales_per_Dollar	Total_Ad_Budget	Sales
0	230.1	37.8	69.2	68.26	11.21	20.53	0.065559	337.1	
1	44.5	39.3	45.1	34.52	30.49	34.99	0.080683	128.9	
2	17.2	45.9	69.3	12.99	34.67	52.34	0.070242	132.4	
3	151.5	41.3	58.5	60.29	16.43	23.28	0.073617	251.3	

Next steps: Generate code with df New interactive sheet

I decided to try and see if making levels for spending will help any analysis I do.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error, r2_score

X = df[["TV Ad Budget ($)", "Radio Ad Budget ($)", "Newspaper Ad Budget ($)"]]
y = df["Sales(Units)"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

predictions = model.predict(X_test)

print("R2 Score:", r2_score(y_test, predictions))
print("MAE:", mean_absolute_error(y_test, predictions))
```

```
R2 Score: 0.899438024100912
MAE: 1.4607567168117603
```

explanation:

The initial linear regression model was created for the prediction of sales using advertising budgets for TV, radio, and newspaper advertising channels. The initial linear regression model showed a good R^2 value, implying that advertising spending was the major driver of sales. Among the advertising channels, the TV advertising channel was the most influential for the predicted sales. The least influential advertising channel was the newspaper advertising channel.

```
from sklearn.tree import DecisionTreeRegressor

dt_model = DecisionTreeRegressor(random_state=42)
dt_model.fit(X_train, y_train)

dt_preds = dt_model.predict(X_test)


dt_r2 = r2_score(y_test, dt_preds)
dt_mae = mean_absolute_error(y_test, dt_preds)

print("Decision Tree R2:", dt_r2)
print("Decision Tree MAE:", dt_mae)
```

```
Decision Tree R2: 0.9310914968293178
Decision Tree MAE: 0.9850000000000001
```

Explanation:

I decided to use the decision tree regressor to add more relalistic patterns in my project. Unlike the linear regression, this adds simple rules into the project like "if tv sales is high then sales is high". this gives this project a more human like approach to the project.

```
from sklearn.ensemble import RandomForestRegressor

rf_model = RandomForestRegressor(random_state=42)
rf_model.fit(X_train, y_train)

rf_preds = rf_model.predict(X_test)


rf_r2 = r2_score(y_test, rf_preds)
rf_mae = mean_absolute_error(y_test, rf_preds)

print("Random Forest R2:", rf_r2)
print("Random Forest MAE:", rf_mae)
```

```
Random Forest R2: 0.9812843792541843
Random Forest MAE: 0.6200999999999988
```

Explanation:

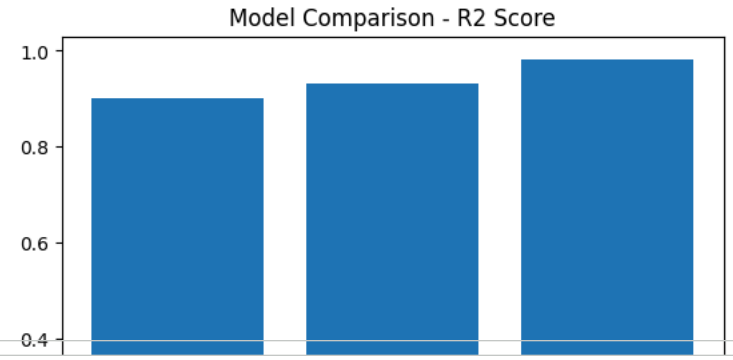
the random forest regressor is an upgraded version of the decision tree by making many decision trees instead of just one tree. It then combines the trees to come up with a more accurate prediction. the multiple trees also reduces the amount of errors in the prediction.

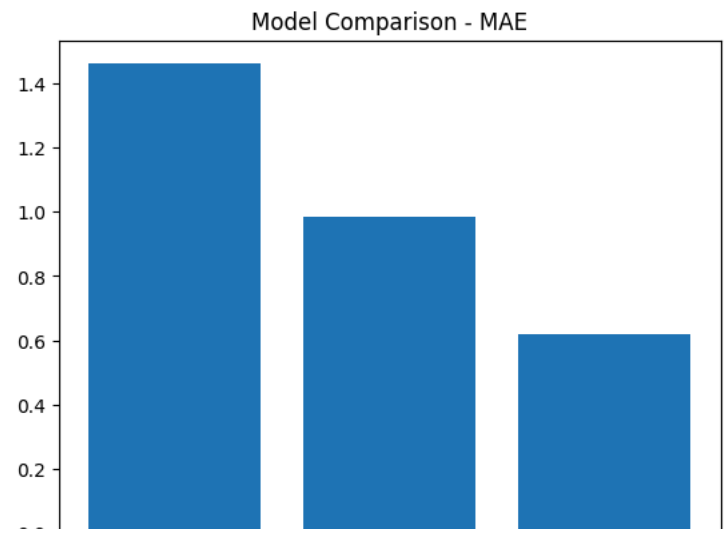
```
from sklearn.metrics import mean_absolute_error, r2_score

results = {
    "Model": ["Linear Regression", "Decision Tree", "Random Forest"],
    "R2": [
        r2_score(y_test, predictions),
        r2_score(y_test, dt_preds),
        r2_score(y_test, rf_preds)
    ],
    "MAE": [
        mean_absolute_error(y_test, predictions),
        mean_absolute_error(y_test, dt_preds),
        mean_absolute_error(y_test, rf_preds)
    ]
}

results_df = pd.DataFrame(results)
print(results_df)
```

```
               Model        R2       MAE
0  Linear Regression  0.899438  1.460757
1      Decision Tree  0.931091  0.985000
2      Random Forest  0.981284  0.620100
```

this showed that the random forest section is by far the best for the MAE and the best model for my R2. this is not that suprising to me as random forest is an upgraded version of the decision tree. so I will be using random forest to do my research.

```
plt.figure()
plt.bar(results_df["Model"], results_df["R2"])
plt.title("Model Comparison - R2 Score")
plt.xticks(rotation=45)
plt.show()
```




```
plt.figure()
plt.bar(results_df["Model"], results_df["MAE"])
plt.title("Model Comparison - MAE")
plt.xticks(rotation=45)
plt.show()
```